Quicksort, part 2

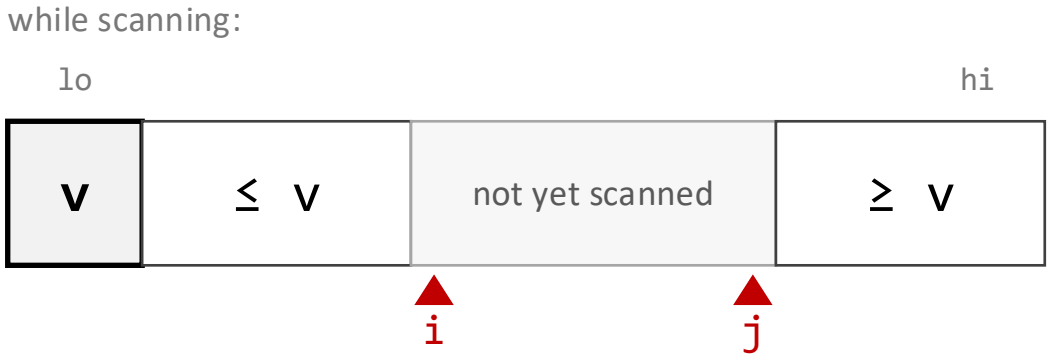
Algorithms §2.3 (and quickselect, from §2.5)

Agenda

- **Recap:** the partition rules from last time
- **Why quicksort is fast:** more compares than mergesort, but less data movement
- **How unlikely is bad luck?** What we gain from shuffling
- **Duplicate keys:** why the book's scans stop on equal items
- **Three-way partitioning:** set the equal items aside for good
- **Quickselect:** find the median without sorting

Last time: partitioning in place

1. The pivot is **v = a[lo]**. i starts just right of it; j starts at the last item, **a[hi]**.
2. **Scan with i:** move right until a[i] is **not less** than v.
3. **Scan with j:** move left until a[j] is **not greater** than v.
4. If i and j have **crossed** (i ≥ j), go to step 5. Otherwise **swap a[i] and a[j]**, step each pointer once more, and go back to step 2.
5. **Swap the pivot a[lo] with a[j].** Now v is in its final position.



Everything between the pivot and i is ≤ v. Everything right of j is ≥ v.

Last time's promise: both pointers **stop on items equal to v**, which looks like wasted work. Today we'll see why it isn't.

More compares, but faster



Table 1

NUMBER OF ITEMS	MERGE SORT	QUICKSORT
500	2 min 8 sec	1 min 21 sec
1,000	4 min 48 sec	3 min 8 sec
1,500	8 min 15 sec*	5 min 6 sec
2,000	11 min 0 sec*	6 min 47 sec

Times reported in C. A. R. Hoare, "Quicksort," The Computer Journal 5(1), 1962, from an Elliott 405 (left).

Quicksort beat mergesort in Hoare's very first measurements, and it usually still does, even though on average it makes **more compares** (~1.39 n lg n, versus at most n lg n). So where does the speed come from?

Look inside the inner loops

Quicksort: partition's scans (lines 5–6 from last time)

```
1  do i = i + 1 while a[i] < v and i < hi
2  do j = j - 1 while a[j] > v and j > lo
```

- Each step: move a pointer, **read one item**, compare it to v (which sits in a variable).
- Items move only when there's a **swap**.

Mergesort: merge

```
1  for k = lo to hi: aux[k] = a[k]
2  for k = lo to hi:
3    if i > mid:   a[k] = aux[j++]
4    elif j > hi:  a[k] = aux[i++]
5    elif aux[j] < aux[i]: a[k] = aux[j++]
6    else:         a[k] = aux[i++]
```

- **Every item** is copied out to aux, then copied back to a.
- That happens at **every level** of the recursion, about lg n times per item.

Compares aren't the only cost. Moving data costs time too, and that's where the two sorts really differ.

Counting the moves

Both sorts on the same 1,000,000 shuffled integers, counted by running the book's code:

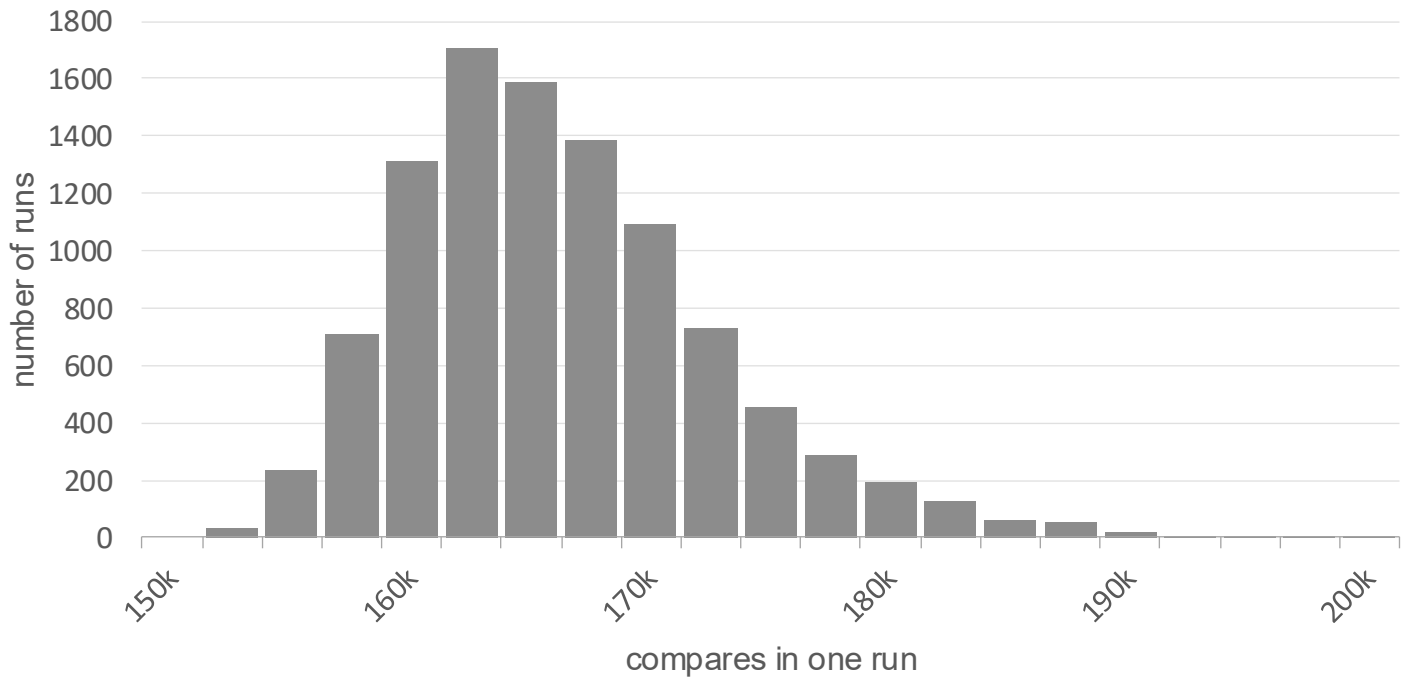
	mergesort	quicksort	quicksort vs mergesort
compares	18,674,044	25,947,699	39% more
array writes (items moved)	39,902,848	9,335,414	**about 4× fewer**

Mergesort's writes: every copy into aux and back into a. Quicksort's writes: 2 per exchange.

Quicksort does **more comparing** but **much less moving**. On real computers that trade usually favors quicksort.

(Real speed also depends on hardware details like memory caches. Counts are the part we can reason about on paper.)

How unlikely is bad luck?



n = 10,000, shuffled 10,000 times

average	167k
worst run	202k
worst case	**50M**

Worst case: $n^2/2 = 50{,}000{,}000$. On this chart, that bar would be **hundreds of feet** off the right edge of the slide.

Out of 10,000 shuffles, the unluckiest run used only **21% more** compares than average. The bigger n gets, the tighter the hump: the spread shrinks compared to the average.

Sedgewick and Wayne put it this way: on a large array, quadratic quicksort is **far less likely than your computer being struck by lightning**.

Two kinds of randomized algorithm

Las Vegas: always right, running time varies

- The randomness only affects **how long** it takes.
- The answer is **always correct**.
- **Quicksort** is one: a bad shuffle makes it slow, never wrong. (So is quickselect, later today.)

Monte Carlo: fast, but can be wrong

- The randomness can affect **the answer**, with a small, known chance of error.
- Example: testing whether a huge number is prime (**Miller–Rabin**), used when making encryption keys.
- One round lets a non-prime slip through at most **¼** of the time. Run 40 independent rounds: at most $(¼)^{40}$, about 1 in 10^{24}.

The names don't mean much, other than being references to cities with gambling-related tourism as a major industry. Monte Carlo methods got their name at Los Alamos in the 1940s; László Babai coined “Las Vegas” in 1979 as its partner.

Duplicate keys are everywhere

- So far our examples had **no repeated keys**: QUICKSORT has nine different letters.
- Real data is full of repeats: a roster sorted by **section**, orders sorted by **state**, events sorted by **date**, survey answers sorted by **yes/no**.
- Often that's **why** you're sorting: to put equal items side by side.
- Today's question: what should partition do when a scan hits an item **equal to the pivot**?

name	section
Adams	2
Baker	1
Chen	2
Diaz	1
Evans	2
Fox	1

Last time's roster: 6 students, but only 2 different keys

Two ways to handle an equal key

Option A: keep going

- i keeps going while a[i] **≤ v**
- j keeps going while a[j] **≥ v**

Equal items never get swapped. Sounds like less work!

Option B: stop (the book's rule)

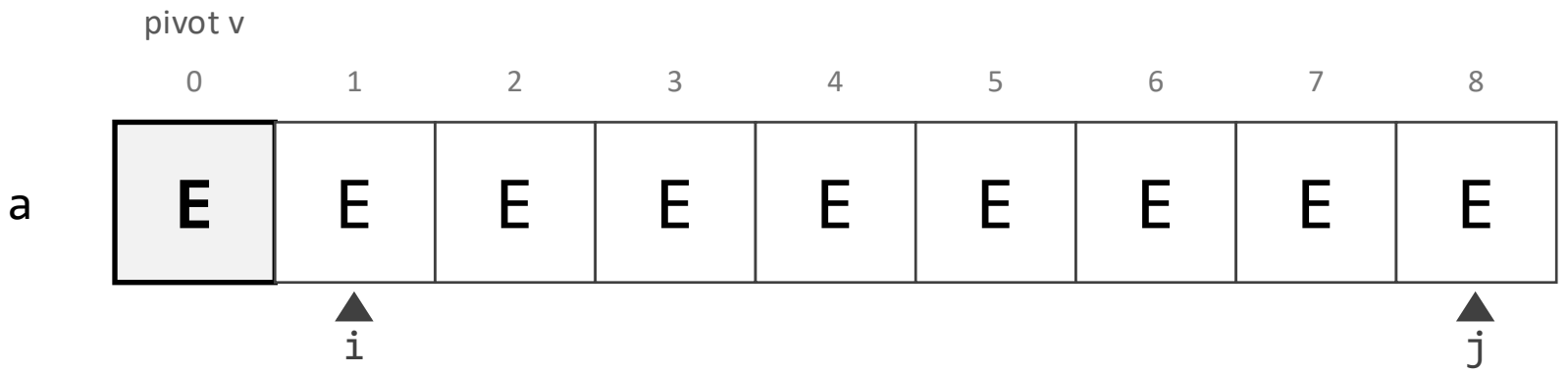
- i stops at an item **not less** than v
- j stops at an item **not greater** than v

This is what you wrote in HW4. Equal items get swapped with each other.

Predict: take an array where **every key is the same**: E E E E E E E E E. Partition it with each option.

Where does i stop? Where does j stop? Where does the pivot end up?

Option A: trace



cost so far

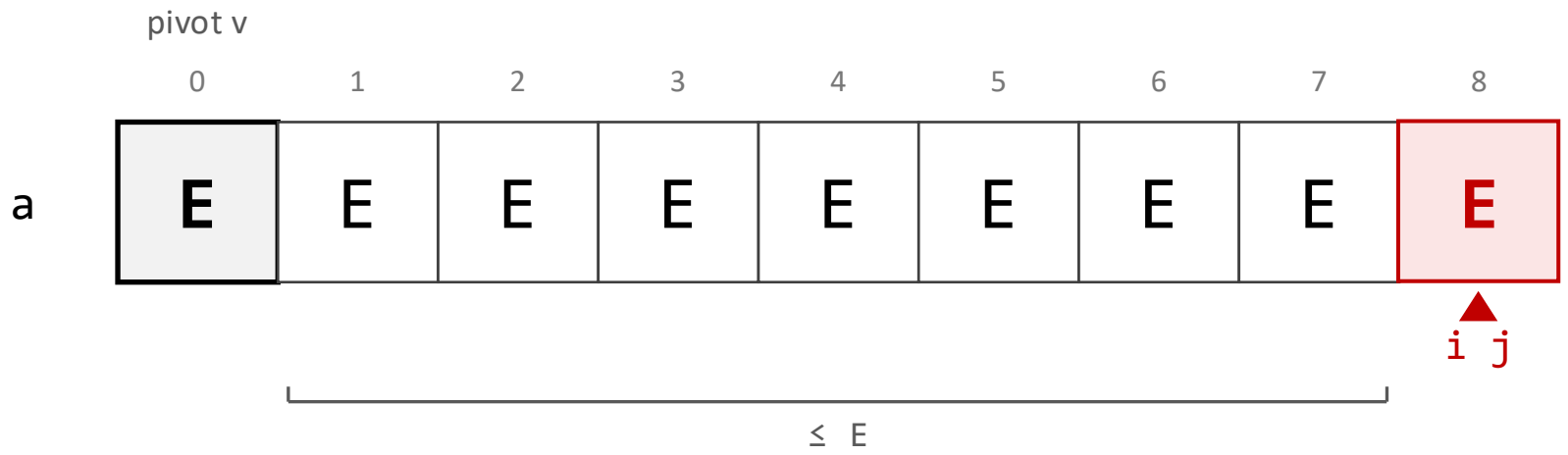
compares	0
exchanges	0

nothing compared yet

Every key is E, so the pivot is E. i starts at a[1], j at a[8]. Option A: keep going past items **equal** to the pivot.

i: step right, keep going while the item is **≤ v** · **j:** step left, keep going while the item is **≥ v** · **crossed?** swap a[lo] with a[j]

Option A: trace



cost so far

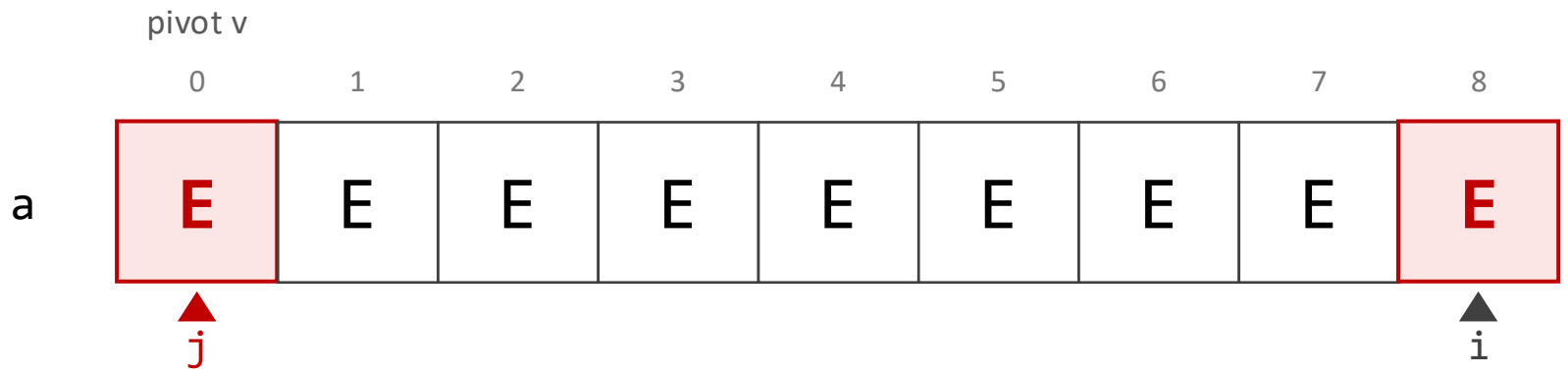
compares	8
exchanges	0

i: E, E, E, E, E, E, E, E

Every item is ≤ E, so **i runs all the way to the end**. It only stops at a[8] because i == hi.

i: step right, keep going while the item is **≤ v** · **j:** step left, keep going while the item is **≥ v** · **crossed?** swap a[lo] with a[j]

Option A: trace



cost so far

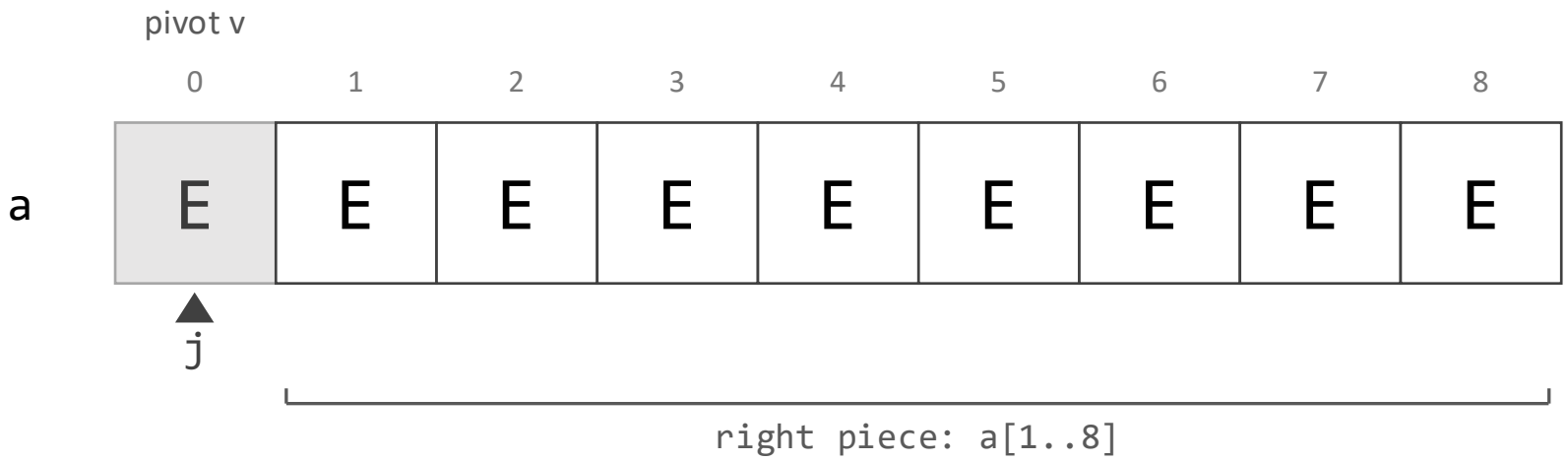
compares	17
exchanges	0

j: E, E, E, E, E, E, E, E, E

Every item is ≥ E too, so **j runs all the way back** to a[0], the pivot itself. It stops there because j == lo. The pointers have crossed.

i: step right, keep going while the item is **≤ v** · **j:** step left, keep going while the item is **≥ v** · **crossed?** swap a[lo] with a[j]

Option A: trace



cost so far

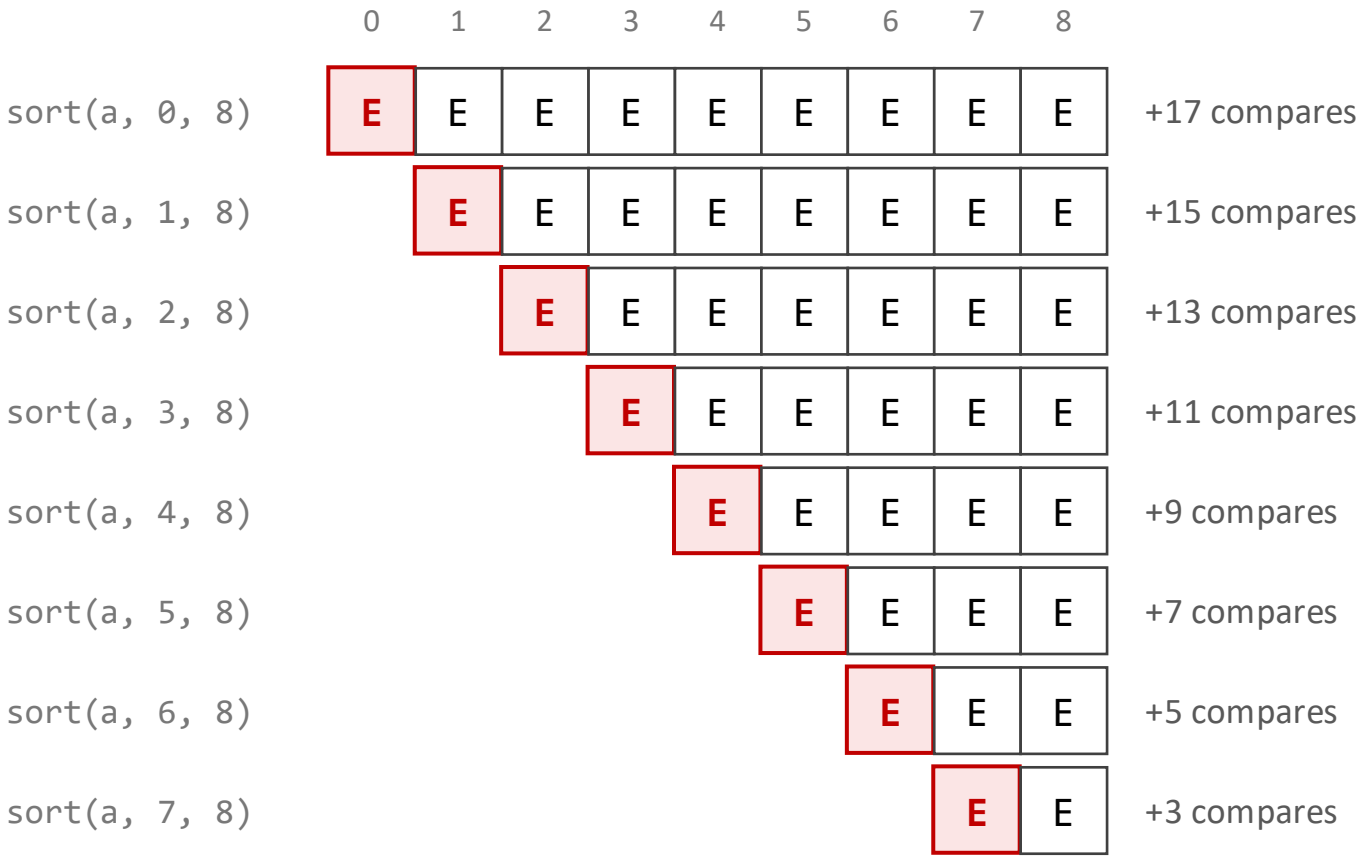
compares	17
exchanges	1

swap pivot into a[j]

Swap the pivot with a[j], which **is** the pivot. The pivot stays at a[0]. Left piece: **empty**. Right piece: a[1..8], only **one item shorter**.

i: step right, keep going while the item is **≤ v** · **j:** step left, keep going while the item is **≥ v** · **crossed?** swap a[lo] with a[j]

Option A: the calls

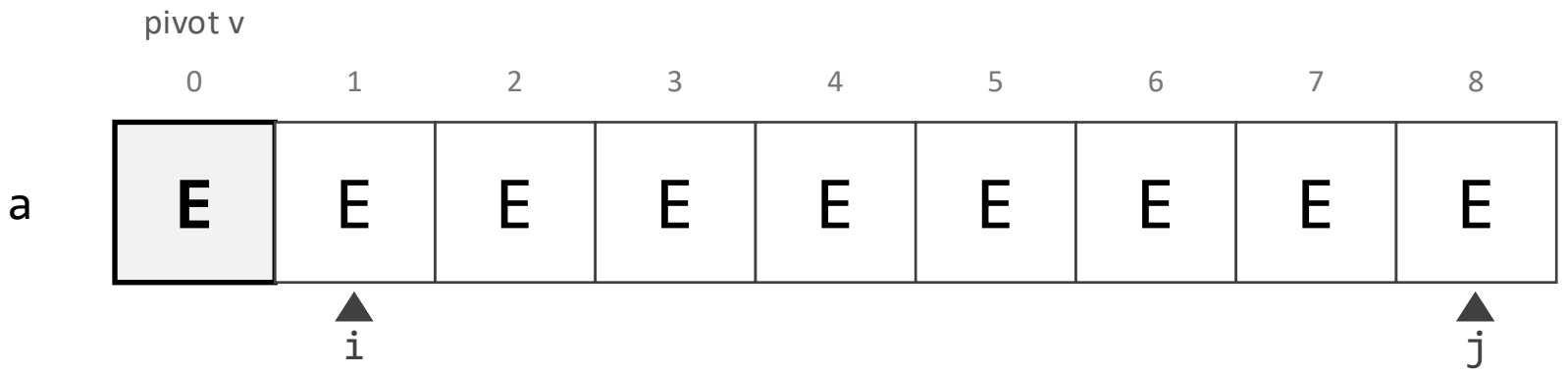


all-equal input, option A

compares	80
exchanges	8

Every pivot lands at the **left end** of its piece, exactly like sorted input with no shuffle. Shuffling can't help: every order of E E E … E looks the same. 9 items, **80 compares**: about **n^2**.

Option B: trace



cost so far

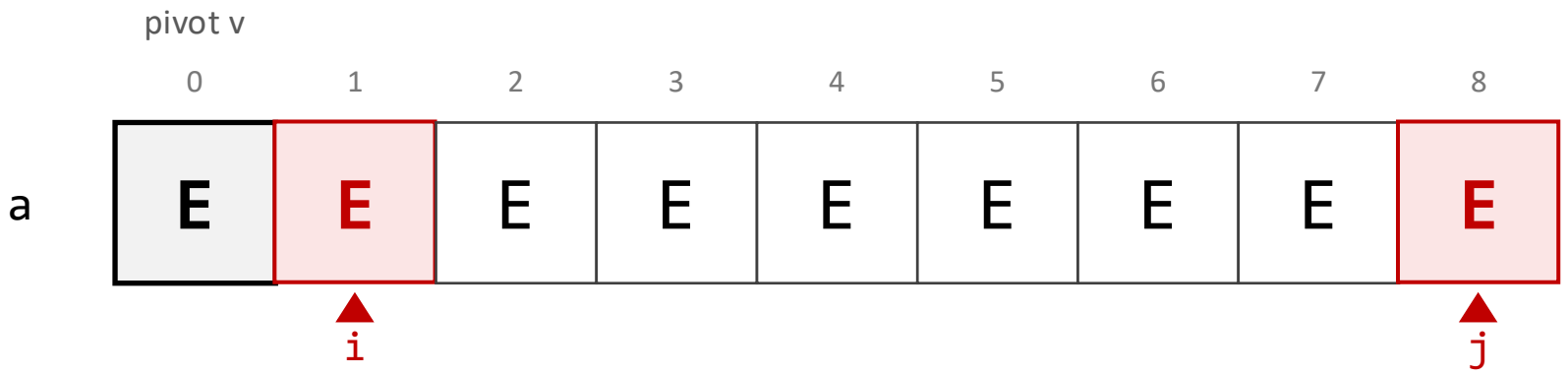
compares	0
exchanges	0

nothing compared yet

Same array, the book's rule this time: **stop** at items equal to the pivot.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Option B: trace



cost so far

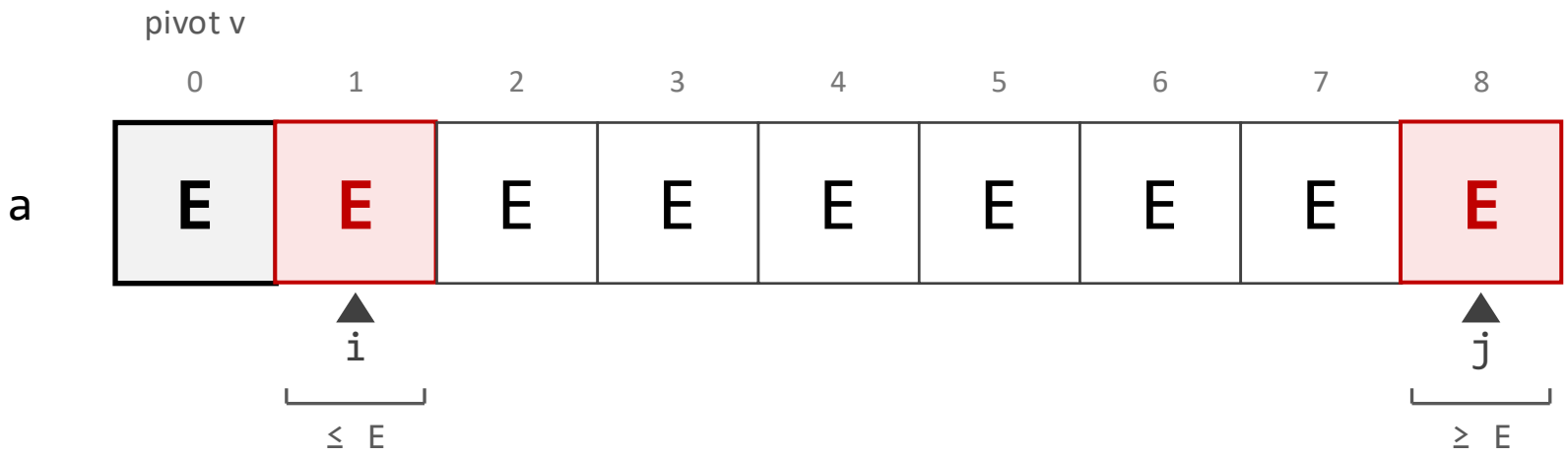
compares	2
exchanges	0

i: E · j: E

i checks a[1]: E is **not less** than E, so i stops right away. j checks a[8]: E is **not greater** than E, so j stops too.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Option B: trace



cost so far

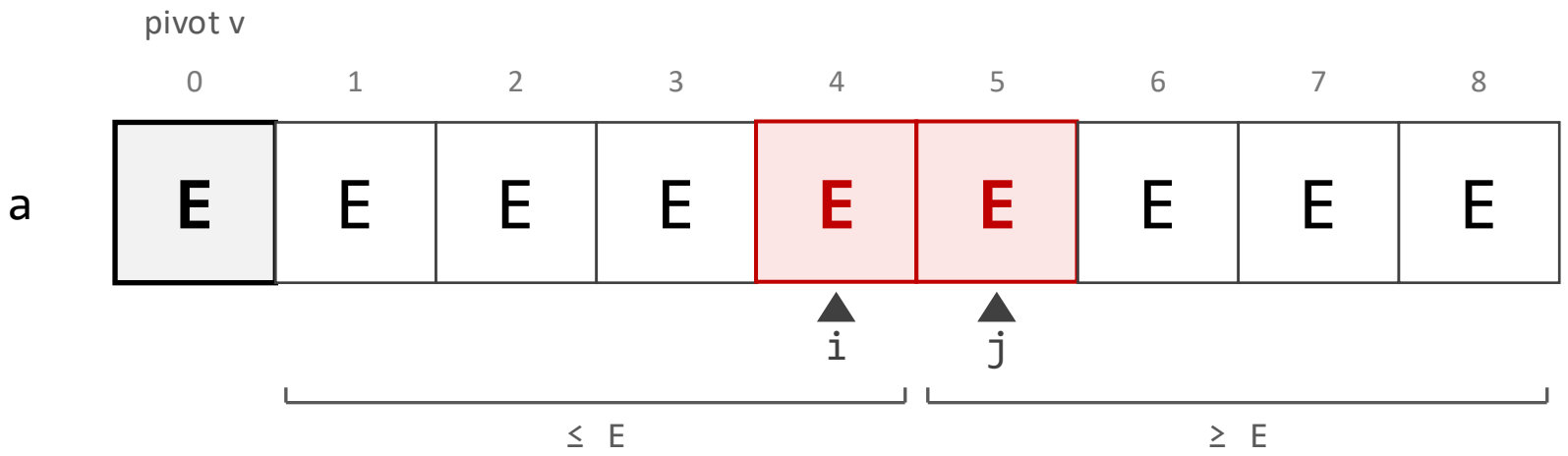
compares	2
exchanges	1

swap a[1] and a[8]

Not crossed, so **swap** them. Swapping an E with an E changes nothing you can see. This is the “wasted work.”

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Option B: trace



cost so far

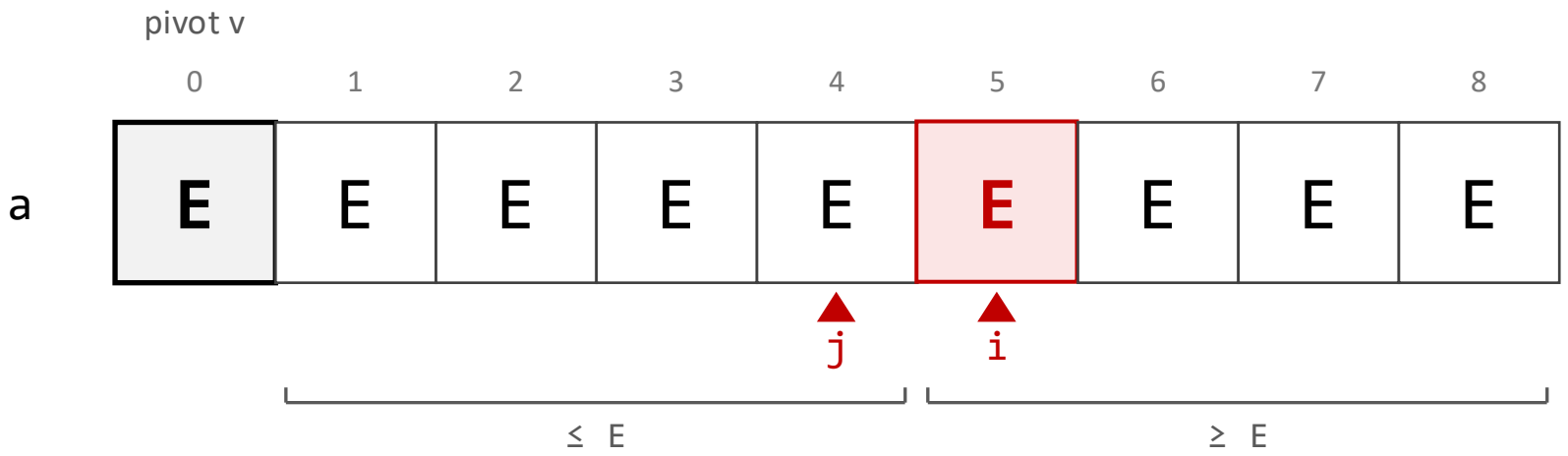
compares	8
exchanges	4

3 more rounds of stop, stop, swap

The same thing happens three more times: stop, stop, swap. Each round, i and j each take **one step toward the middle**.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Option B: trace



cost so far

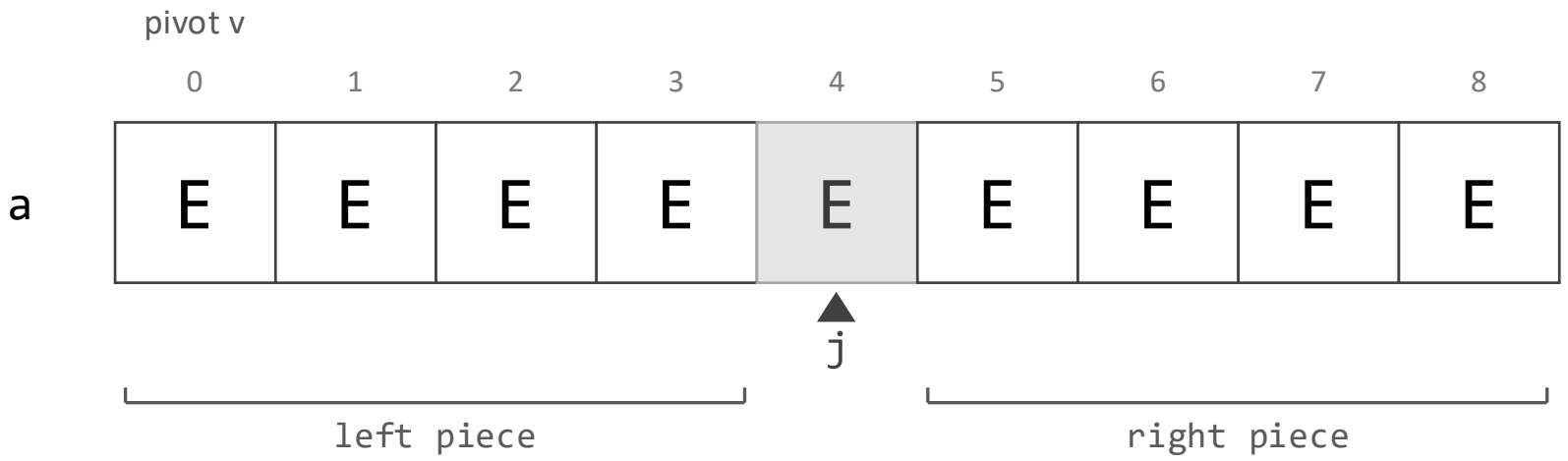
compares	**10**
exchanges	4

i: E · j: E

i stops at a[5], j stops at a[4]. Now j is left of i: **crossed**. They met **in the middle**.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Option B: trace



cost so far

compares	10
exchanges	5

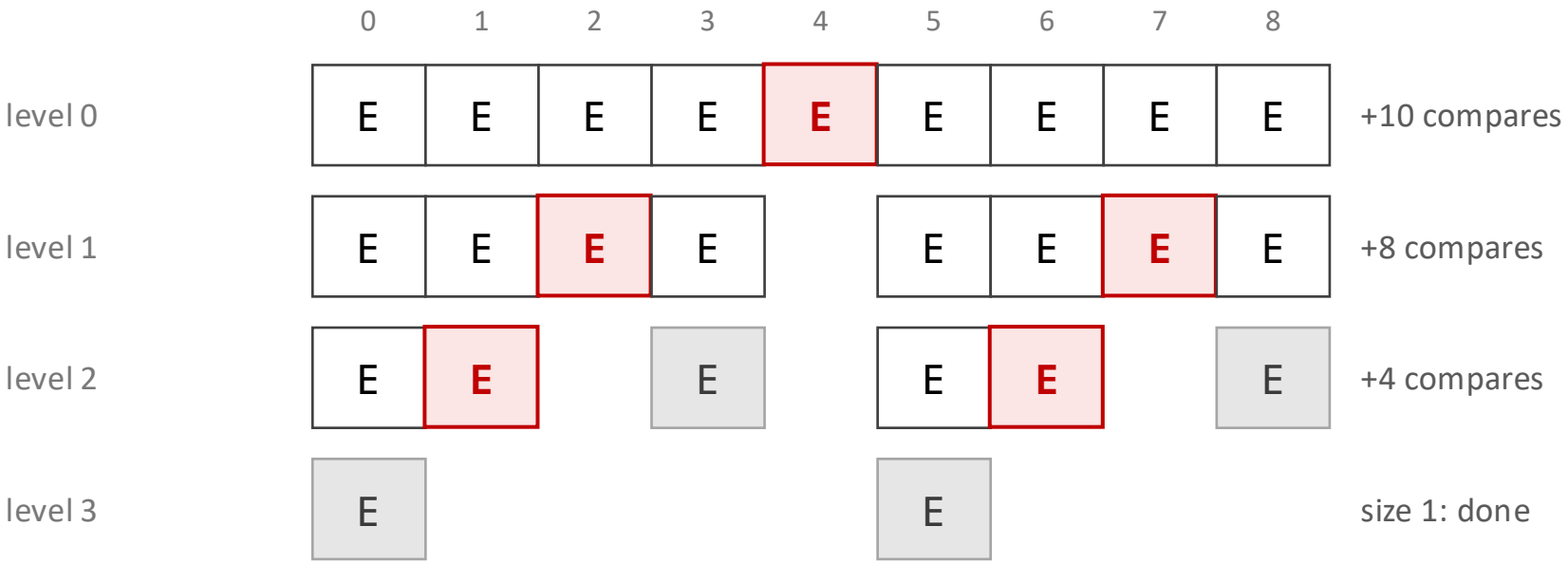
swap pivot into a[j]

Swap the pivot with a[4]. The pivot lands **in the middle**, and the two pieces are **halves**: a[0..3] and a[5..8]. That's the best split quicksort can get.

i: step right, stop at an item **not less** than v · **j:** step left, stop at an item **not greater** than v · **crossed?** swap a[lo] with a[j]

Option B: the calls

Each row is one level of recursion. Red: where that call's pivot landed.



all-equal input, option B

compares	22
exchanges	11

Pieces **halve** at every level, just like mergesort. Option B: **22 compares**. Option A: **80**. In general, option B costs about **n lg n**; option A about **n^2**.

Option A vs. option B, measured

Compares, counted by running each version once on shuffled input:

input	n	option A: keep going	option B: stop (book)
every key the same	10,000	**99,999,999**	121,034
every key the same	100,000	**9,999,999,999**	1,561,130
random 0s and 1s	100,000	**3,764,387,389**	1,581,578
all keys different	100,000	2,085,421	2,085,421

With lots of duplicates, option A goes **quadratic**: about 2,400 times more compares on 0s and 1s. With no duplicates, the two options are identical.

So the book's “wasted” swaps are what keep the pieces balanced. **That's** why both pointers stop on equal keys.

This bug shipped in a real library

- Jon Bentley and Doug McIlroy of Bell Labs tested the `qsort` function in the C libraries of a dozen Unix systems (paper published 1993).
- The Berkeley Unix version took **quadratic time on random 0s and 1s**, “precisely because it scans over equal elements.” That's option A.
- They found no library `qsort` that couldn't easily be pushed into quadratic time, and they wrote a replacement that many libraries adopted.

Source: J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” Software: Practice and Experience 23(11), 1993.

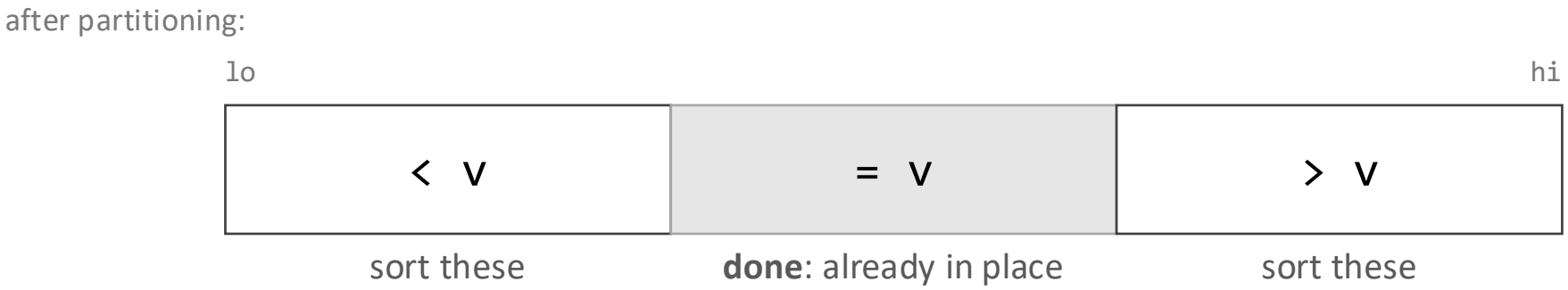
Their fix for equal keys was a version of the idea coming up next: split the array into **three** parts instead of two.

Three-way partitioning



Option B still does about n lg n work on an array of all-equal keys, re-partitioning items that are already in order. Can we do better?

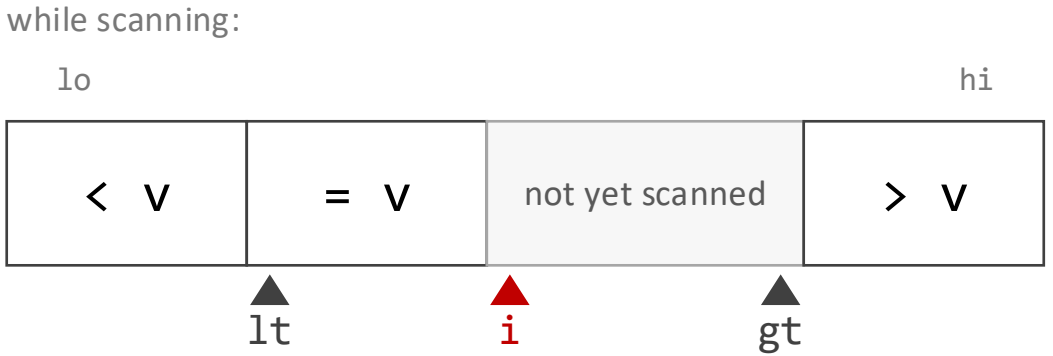
Idea (Dijkstra's “Dutch national flag” problem): partition into **three** groups, and **never touch the middle group again**.



If every key is equal, the “< v” and “> v” groups are empty after one pass, and we're done: **n – 1 compares** in total.

Three-way partitioning: the rules

1. The pivot value is **v = a[lo]**. Start with **lt = lo**, **i = lo + 1**, **gt = hi**.
2. Look at a[i] and compare it with v:
   - **less:** swap a[lt] and a[i]. Then **lt++** and **i++**.
   - **greater:** swap a[i] and a[gt]. Then **gt--**. Don't move i: the item that just arrived hasn't been looked at.
   - **equal:** just **i++**.
3. Repeat until i passes gt (**i > gt**). Then sort a[lo..lt-1] and a[gt+1..hi].



a[lo..lt-1] < v, a[lt..i-1] = v, a[gt+1..hi] > v.

Every step shrinks the gray middle by one, so a piece of size n takes **n – 1 compares** to partition.

Three-way partitioning: trace



cost so far

compares	0
exchanges	0

nothing compared yet

"Dutch flag" letters: **R**ed, **W**hite, **B**lue. The pivot value is **v = R**. **lt** = 0, **i** = 1, **gt** = 8. Gray: not yet scanned.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

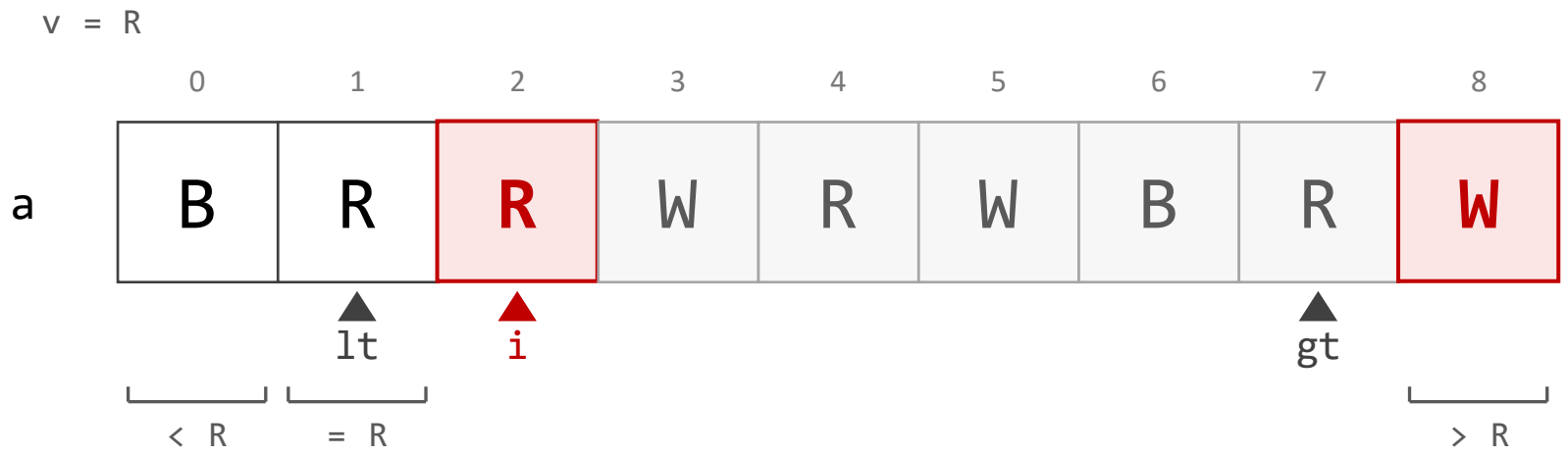
compares	**1**
exchanges	**1**

swap a[lt] and a[i]

a[i] was **B**, less than R: **swap a[lt] and a[i]**, then lt++ and i++. That swap moved the R: three-way partitioning **does** swap equal keys.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

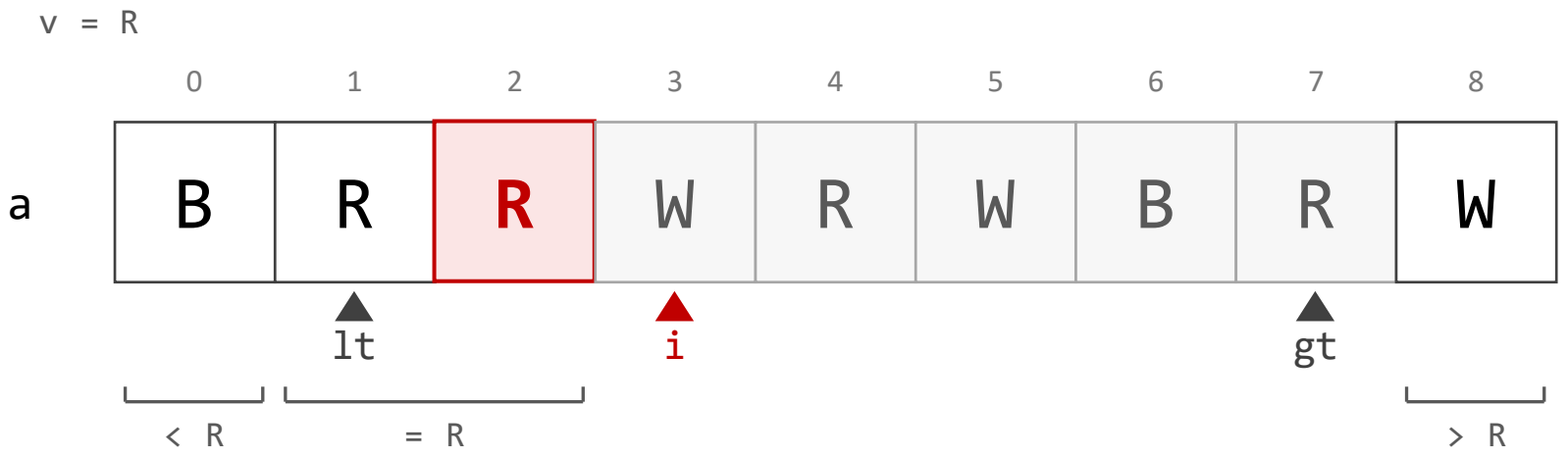
compares	**2**
exchanges	**2**

swap a[i] and a[gt]

a[i] was **W**, greater than R: **swap a[i] and a[gt]**, then gt--. **i stays put:** the R that just arrived from the right hasn't been checked yet.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

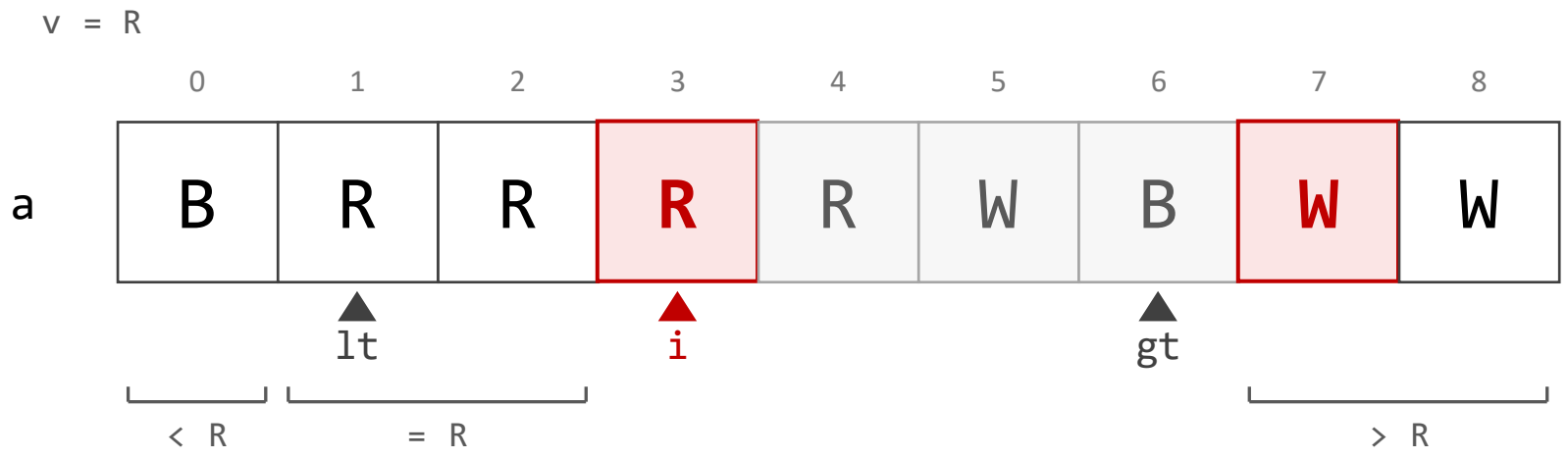
compares	**3**
exchanges	2

equal: i++

a[i] is **R**, equal to the pivot: just **i++**. The middle group grows by one.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

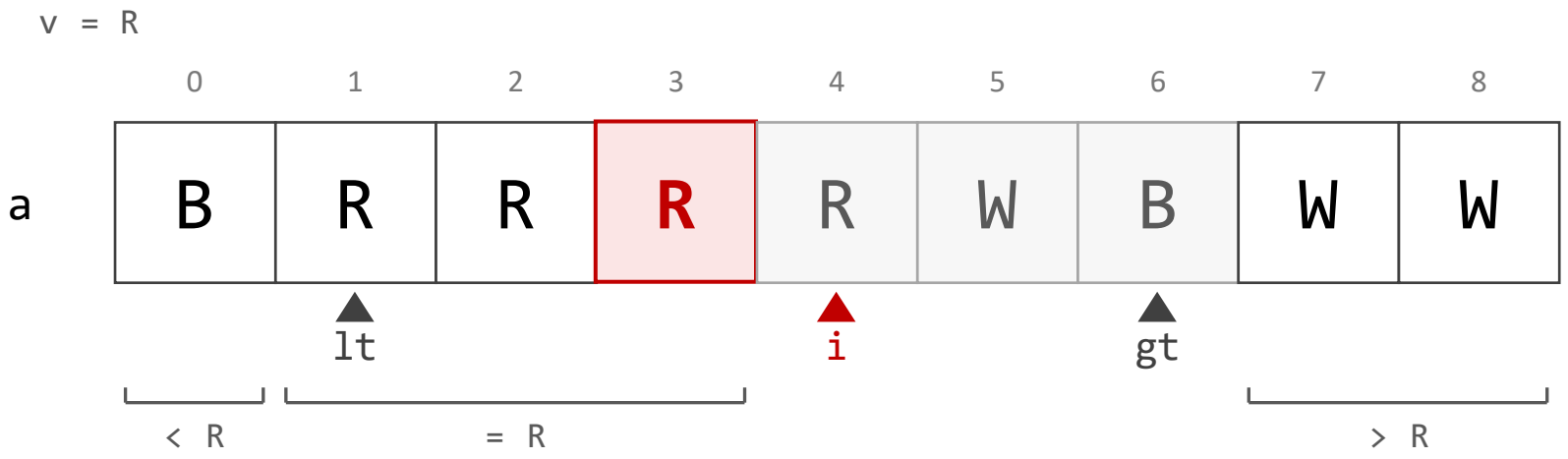
compares	4
exchanges	3

swap a[i] and a[gt]

a[i] was **W**, greater than R: **swap a[i] and a[gt]**, then gt--. **i stays put:** the R that just arrived from the right hasn't been checked yet.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

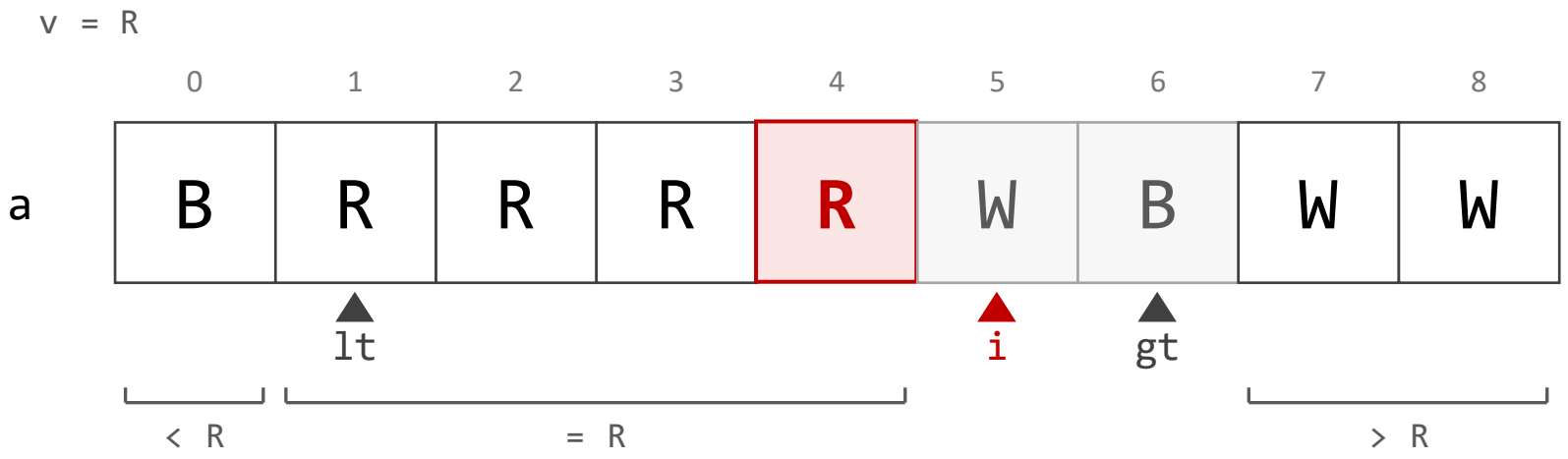
compares	**5**
exchanges	3

equal: i++

a[i] is **R**, equal to the pivot: just **i++**. The middle group grows by one.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

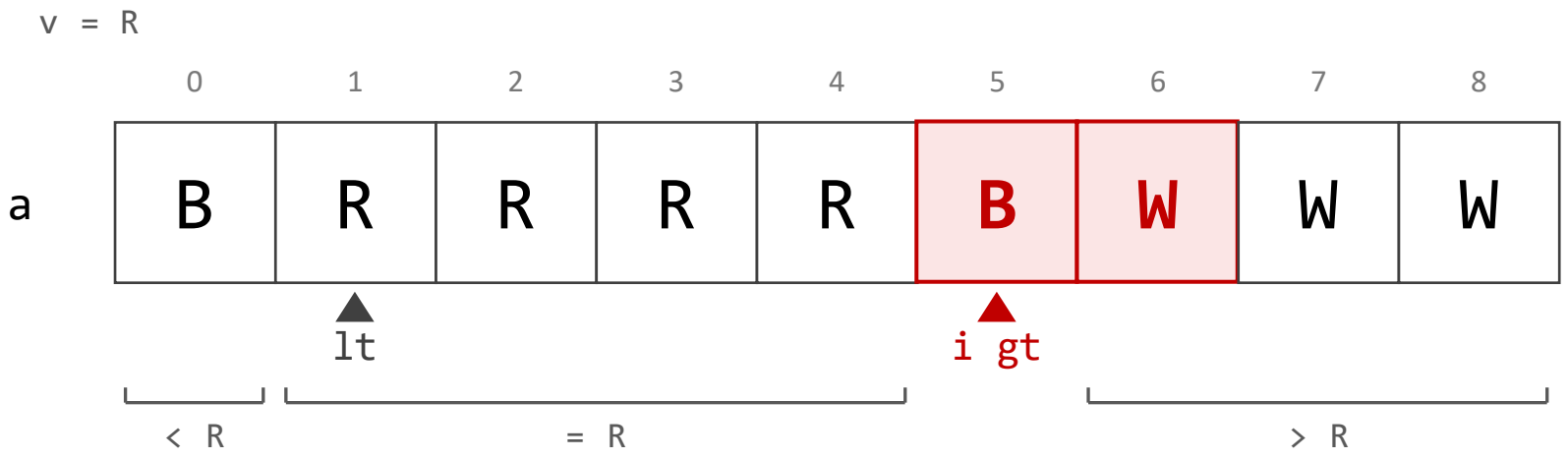
compares	**6**
exchanges	3

equal: i++

a[i] is **R**, equal to the pivot: just **i++**. The middle group grows by one.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

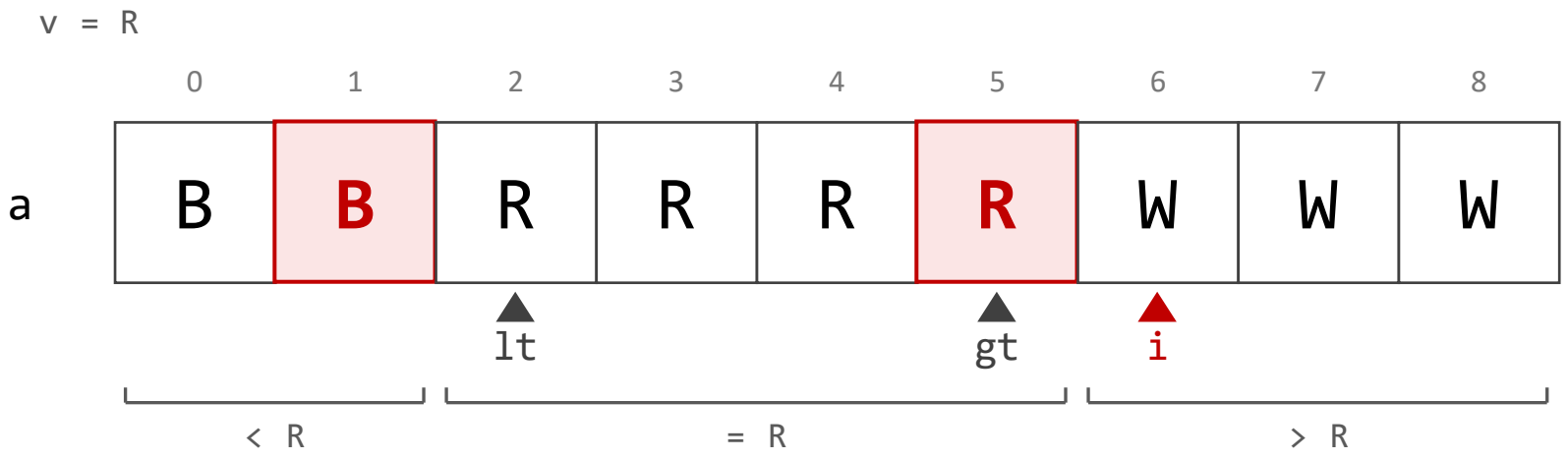
compares	7
exchanges	4

swap a[i] and a[gt]

a[i] was **W**, greater than R: **swap a[i] and a[gt]**, then gt--. **i stays put:** the B that just arrived from the right hasn't been checked yet.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: trace



cost so far

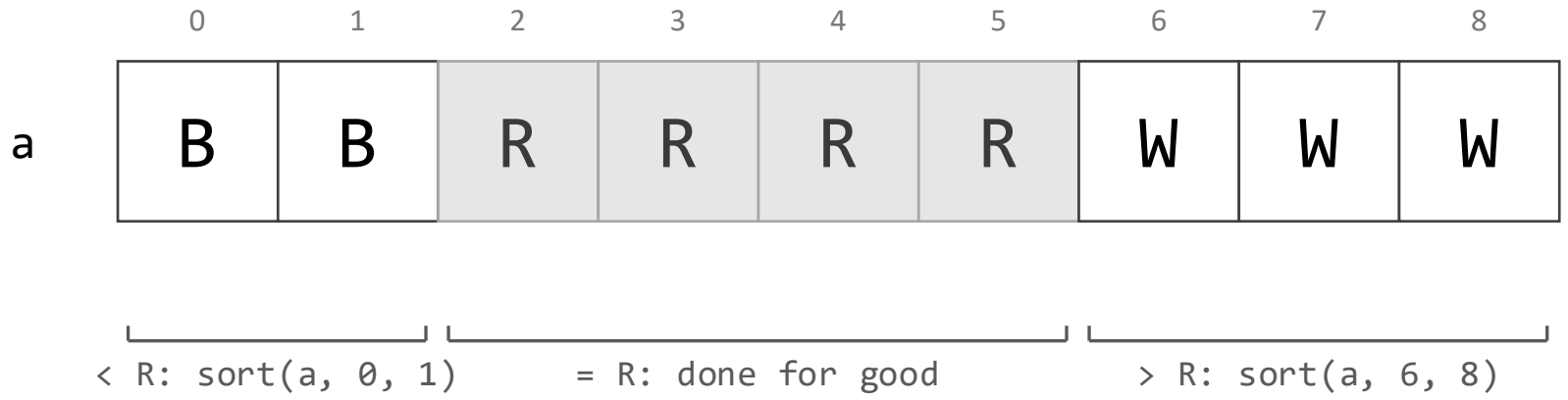
compares	8
exchanges	5

swap a[lt] and a[i]

a[i] was **B**, less than R: **swap a[lt] and a[i]**, then lt++ and i++. Now i > gt: **the gray region is empty**, so partitioning is done.

a[i] < v: swap a[lt], a[i]; lt++, i++ · **a[i] > v:** swap a[i], a[gt]; gt-- · **a[i] = v:** i++ · **stop** when i > gt

Three-way partitioning: done



cost so far

compares	8
exchanges	5

n − 1 = 8 compares

One pass, **8 compares**. All four R's are in their final positions **at once**, and neither recursive call will ever look at them again. With 2-way partitioning, only the pivot itself is guaranteed to be final.

Three-way quicksort: pseudocode

```
1  sort3(a, lo, hi):
2      if hi <= lo: return                                  (one item or none)
3      v = a[lo],  lt = lo,  i = lo + 1,  gt = hi
4      while i <= gt:                                       (gray region not empty)
5          c = compare(a[i], v)                             (one compare per step)
6          if c < 0:   swap a[lt] and a[i];  lt++;  i++
7          elif c > 0: swap a[i] and a[gt];  gt--            (i stays put)
8          else:       i++
9      sort3(a, lo, lt - 1)                                 (the < v group)
10     sort3(a, gt + 1, hi)                                 (the > v group)
```

- There's **no separate partition method** and no final pivot swap: the pivot moves along with the other R's.
- In Java, `compare` is one call to `compareTo`, which tells you less, equal, or greater all at once.
- The shuffle at the start is the same as before.

Is three-way always better?

n = 100,000, shuffled, one run each:

input	book (2-way): compares	3-way: compares	book: exchanges	3-way: exchanges
every key the same	1,561,130	**99,999**	780,565	**0**
3 different keys (like sections)	1,616,822	**199,738**	732,127	99,741
10 different keys	1,607,818	**330,176**	686,433	230,186
all keys different	2,085,421	2,163,792	393,048	2,163,792

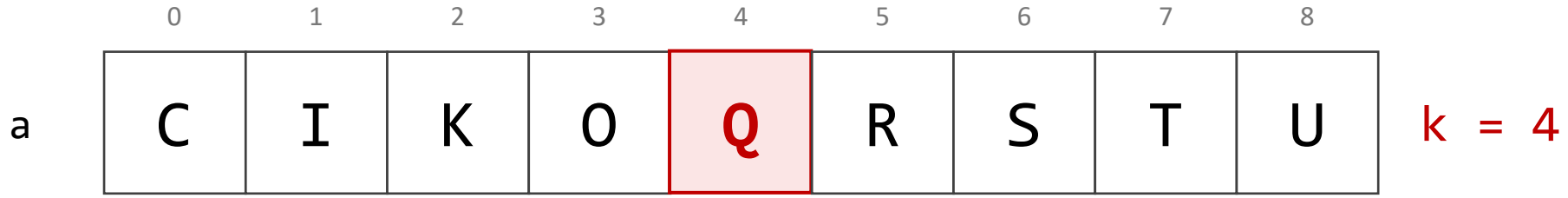
With duplicates, three-way wins **big**. With all keys different, it makes about the same compares but **more than 5× as many exchanges**, since nearly every step is a swap.

So library sorts use cleverer versions that are fast either way (Bentley and McIlroy's, for one). Our versions are the clear ones to learn from.

Quickselect: the problem

- Sometimes you don't need everything sorted. You need **one** item: the **median**, the 10th smallest, the top 100.
- **Selection:** given an array and a number k, find the item that would be at **index k** if the array were sorted.
- Sorting first works, but costs ~1.39 n lg n compares. Can we skip most of that work?

Last time's letters, sorted. The median (k = 4) is Q:



Quickselect: the idea

After partitioning, the pivot is at its **final index j**, smaller items on its left and bigger on its right.

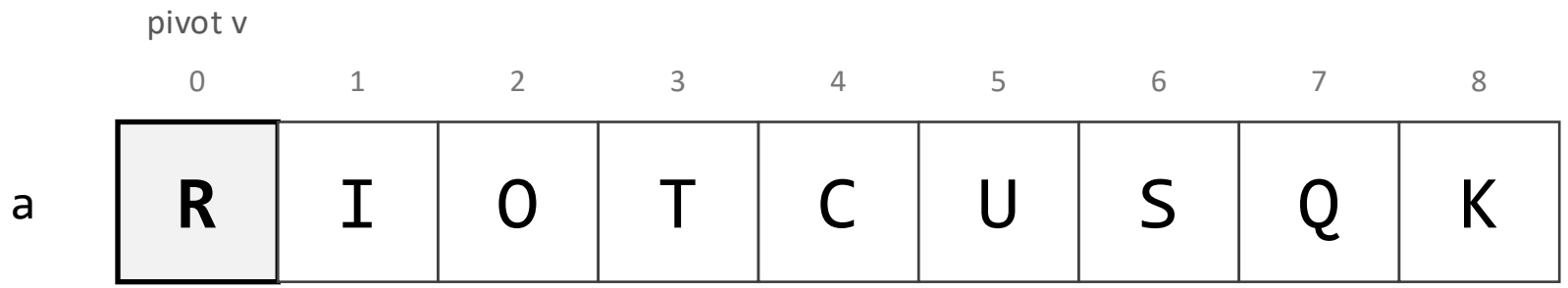
Compare j with k:

- **j == k:** a[j] is the answer. Done!
- **k < j:** the answer is in the **left** piece. Ignore the right piece completely.
- **k > j:** the answer is in the **right** piece. Ignore the left piece completely.

Repeat on the one piece that's left. Quicksort recurses into **both** pieces; quickselect follows **only one**.

Note k is an **index**, so it stays the same the whole time. No adjusting k when you move to the right piece.

Quickselect: trace



cost so far

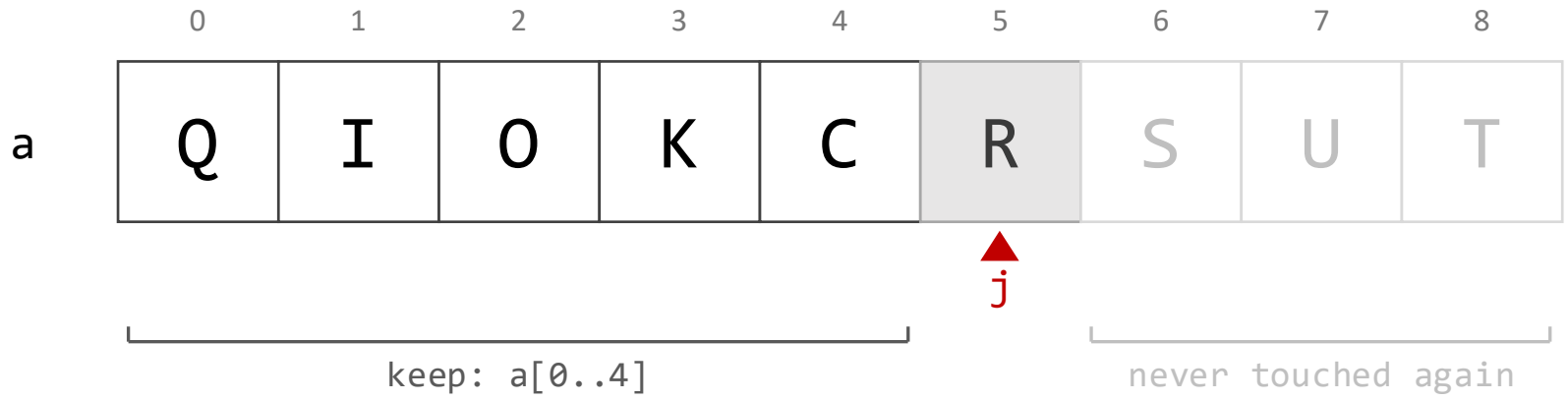
compares	0
exchanges	0

looking for k = 4

Find the median (**k = 4**) of last time's shuffled letters. Quickselect uses **exactly the same partition** as quicksort.

j == k: done · **k < j:** keep only the left piece · **k > j:** keep only the right piece

Quickselect: trace



cost so far

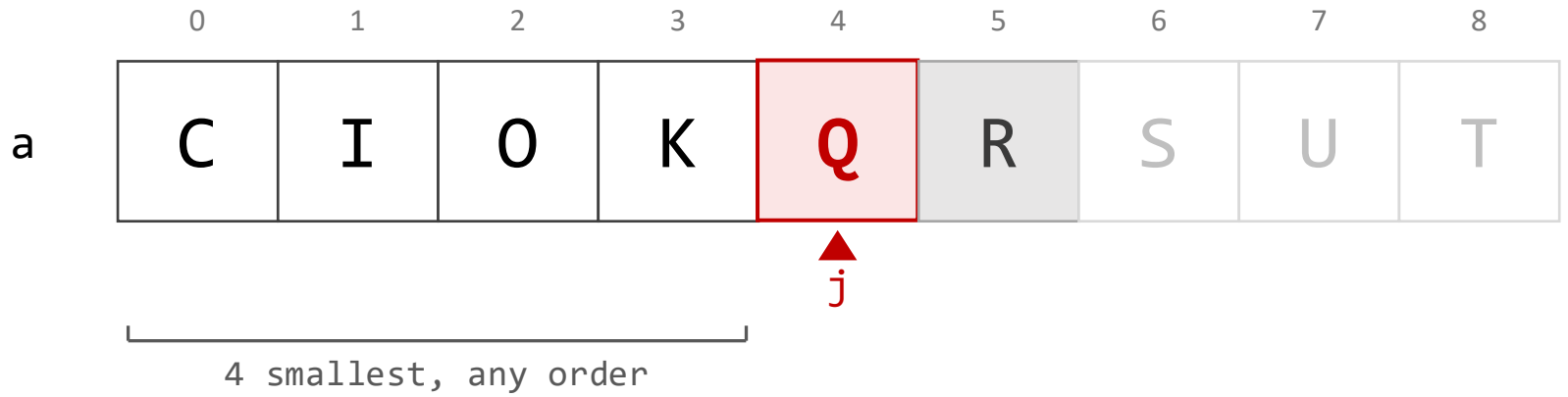
compares	**10**
exchanges	**3**

partition 1 (same as last time)

Partition 1 is last time's: pivot R lands at **j = 5**. k = 4 is less than 5, so the median is in the **left piece**. The right piece is never looked at again.

j == k: done · **k < j:** keep only the left piece · **k > j:** keep only the right piece

Quickselect: trace



cost so far

compares	**15**
exchanges	**4**

partition 2: j == k

Partition a[0..4]: pivot Q lands at **j = 4 = k**. Done: **Q is the median**. 2 partitions, **15 compares**. The full sort took 7 partitions and 32 compares.

Bonus: a[0..3] now holds the 4 smallest items, though not in sorted order.

j == k: done · **k < j:** keep only the left piece · **k > j:** keep only the right piece

Quickselect: pseudocode

```
1  select(a, k):
2      shuffle(a)                                   (same reason as quicksort)
3      lo = 0,  hi = n - 1
4      while hi > lo:
5          j = partition(a, lo, hi)                 (the same partition)
6          if j == k: return a[k]                   (found it)
7          else if j > k: hi = j - 1                (answer is left of j)
8          else: lo = j + 1                         (answer is right of j)
9      return a[k]                                  (piece of size 1)
```

- **No recursion needed:** there's only one piece to follow, so a loop that moves lo or hi does the job. This is the book's **select** (§2.5).
- **Side effect:** the array gets rearranged. a[k] is the answer, everything left of it is ≤ a[k], and everything right of it is ≥ a[k].

Quickselect: about linear time

Intuition: if every pivot landed in the middle, the pieces would be n, n/2, n/4, … and the total work about **n + n/2 + n/4 + … ≈ 2n**.

n	quickselect (median)	per item	full quicksort	sorting costs…
1,000	3,339	3.3	12,151	3.6× as much
10,000	33,424	3.3	167,471	5.0× as much
100,000	338,367	3.4	2,137,212	6.3× as much
1,000,000	3,473,956	3.5	26,086,741	**7.5× as much**

Average compares on shuffled input. Quickselect: 500 runs per row (50 at n = 1,000,000). Quicksort: 10,000 runs per row (1,000 at n = 100,000; 100 at n = 1,000,000).

Real pivots don't land exactly in the middle, so the average for the median is about **3.4n**, not 2n. But it's still **linear**: double n, double the work. Sorting's cost grows faster.

The worst case is still about $n^2/2$, and the shuffle makes it just as unlikely as for quicksort.

Today in one table

	every key the same	few different keys	all keys different
2-way, keep going on equal keys	**~n^2**	**~n^2**	~1.39 n lg n
2-way, stop on equal keys (book, HW4)	~n lg n	~n lg n	~1.39 n lg n
3-way (Quick3way)	**n – 1**	**far below n lg n**	~1.39 n lg n, more swaps

- **Quicksort is fast** because it moves data much less than mergesort, even though it compares more.
- **The shuffle** makes bad luck vanishingly rare; quicksort is a **Las Vegas** algorithm.
- **Quickselect** finds the k-th smallest item in about linear time on average, by following only one piece.

Next time: priority queues (§2.4).

Check yourself

1. Use the book's partition (stop on equal keys) on **A A A A A** (lo = 0, hi = 4). At what index does the pivot end up?

2. Three-way partitioning on **M A Z M Q**, with v = M, lt = 0, i = 1, gt = 4. Show the array, lt, i, and gt after **two** steps.

3. In quickselect you're looking for k = 2, and partition returns j = 6. What are the new lo and hi?

Try them on paper before you go to the next slide.

Check yourself: answers

1. **Index 2, the middle.** i and j stop at every item: swap a[1] and a[4], swap a[2] and a[3], then they cross with j = 2. (6 compares, 3 exchanges, counting the pivot swap.)

2. **A M Q M Z, with lt = 1, i = 2, gt = 3.** Step 1: A < M, so swap a[0] and a[1], then lt++ and i++. Step 2: Z > M, so swap a[2] and a[4], then gt--. i stays at 2, because Q hasn't been checked yet.

3. **lo stays the same, hi = 5.** k < j, so the answer is in the left piece, a[lo..j-1]. k stays 2.